



GUIA LINHA A LINHA

display.py — *Análise Técnica Completa*

`len · // · half_block · cx · cy · lx · lw · lh · thick · abs · bits`

Cada variável explicada com diagramas e geometria



Secção 1 O Grid — A Estrutura Base de Tudo

O que é o self.grid?

O grid é uma lista de listas de inteiros. Cada inteiro representa uma célula do labirinto. Olha para ele como uma folha de papel quadriculada onde cada quadrado tem um número escrito dentro.

```

self.grid = [
    [15, 2, 15, 2, 15, 15, 15], # linha 0 (row 0)
    [15, 0, 15, 0, 5, 0, 15],   # linha 1 (row 1)
    [15, 15, 15, 12, 15, 15, 15], # linha 2 (row 2)
    [12, 0, 15, 0, 15, 0, 6],   # linha 3 (row 3)
    [12, 4, 15, 4, 15, 15, 15], # linha 4 (row 4)
]

```

15	2	15	2	15	15	15
15	0	15	0	5	0	15
15	15	15	12	15	15	15
12	0	15	0	15	0	6
12	4	15	4	15	15	15

Visualização do grid: 5 linhas (rows) × 7 colunas (cols). Cinzento = valor 15 (parede total). Azul = outros valores.

len(self.grid) — contar as LINHAS

```
len(self.grid) → 5 # há 5 linhas no grid acima
```

len() é a função nativa do Python para contar o número de elementos numa lista. Como self.grid é uma lista de listas, len(self.grid) conta quantas listas existem dentro — ou seja, quantas linhas tem o labirinto.



Analogia: imagina que tens uma caixa com 5 gavetas. len(self.grid) diz-te que há 5 gavetas. Não te diz o que está dentro de cada gaveta.

len(self.grid[0]) — contar as COLUNAS

```

self.grid[0] → [15, 2, 15, 2, 15, 15, 15] # a primeira linha
len(self.grid[0]) → 7 # essa linha tem 7 elementos = 7 colunas

```

self.grid[0] acede ao índice 0 da lista principal, ou seja, a primeira linha. Depois len() conta quantos elementos essa linha tem — isso é o número de colunas do labirinto.



Porquê usar [0] e não outra linha? Assumimos que todas as linhas têm o mesmo número de colunas (labirinto retangular). A linha 0 é apenas a referência mais óbvia para medir a largura.



Secção 2 Geometria da Janela: window_width e window_height

self.block_size = 64

Cada célula do grid ocupa um bloco quadrado de 64×64 píxeis no ecrã. É a unidade base de toda a geometria do display. Podes pensar nisto como a 'escala' do mapa.

64 px	64 px	64 px
64 px	64 px	64 px

Cada célula do grid = um bloco de 64×64 píxeis no ecrã

window_width = len(self.grid[0]) * self.block_size

```
len(self.grid[0])    → 7    # 7 colunas no labirinto
self.block_size      → 64    # cada célula = 64 píxeis de largura

window_width = 7 * 64 = 448 píxeis
```

Multiplicamos o número de colunas pela largura de cada bloco. O resultado é a largura total da janela em píxeis. Se o mapa tiver 7 células de largura e cada uma ocupa 64px, a janela precisa de ter 448px de largura.

window_height = len(self.grid) * self.block_size

```
len(self.grid)       → 5    # 5 linhas no labirinto
self.block_size      → 64    # cada célula = 64 píxeis de altura

window_height = 5 * 64 = 320 píxeis
```

Mesma lógica mas para a altura: linhas × altura_por_bloco = altura total da janela.



Isto é o que torna o display dinâmico — se o backend gerar um mapa 20×20, a janela ajusta-se automaticamente para 1280×1280 sem alterar uma linha de código.



Secção 3 draw_rect — O Pincel Fundamental

Assinatura completa

```
def draw_rect(self, start_x: int, start_y: int,
               width: int, height: int, color: int) -> None:
    for y in range(start_y, start_y + height):
        for x in range(start_x, start_x + width):
```

```
self.m.mlx_pixel_put(self.mlx_ptr, self.win_ptr, x, y, color)
```

O sistema de coordenadas do ecrã

O ecrã usa um sistema de coordenadas onde (0, 0) fica no canto superior esquerdo. x cresce para a direita. y cresce para baixo. Isto é o oposto da geometria clássica onde y cresce para cima.

(0,0)	x →	x →	x →	
y ↓				
y ↓		draw_rect		
y ↓		start_x,y		
				(w,h)

Sistema de coordenadas: (0,0) no canto superior esquerdo

O que cada parâmetro controla

Variável	Exemplo (7×5)	O que significa
start_x	col * 64	Píxel X do canto superior esquerdo do retângulo
start_y	row * 64	Píxel Y do canto superior esquerdo do retângulo
width	block_size	Largura em píxeis do retângulo a pintar
height	block_size	Altura em píxeis do retângulo a pintar
color	0xFFFFFFFF	Cor em formato ARGB hexadecimal (A=opacidade, R, G, B)

Os dois ciclos for aninhados

```
for y in range(start_y, start_y + height): # percorre linha a linha (vertical)
    for x in range(start_x, start_x + width): # percorre píxel a píxel (horizontal)
        mlx_pixel_put(x, y, color)          # pinta este píxel
```

range(start_y, start_y + height) gera todos os valores inteiros de start_y até start_y+height-1. O ciclo exterior percorre as linhas (y), o interior percorre os píxeis dentro de cada linha (x). Juntos pintam um retângulo sólido.

💡 Exemplo concreto: draw_rect(10, 20, 30, 5, cor) pinta 30 píxeis de largura × 5 de altura, começando na posição (10, 20). São 150 chamadas a mlx_pixel_put.



O código completo da Layer 1

```
thick = self.wall_thickness    # = 4 píxeis

for row in range(len(self.grid)):
    for col in range(len(self.grid[row])):
        if self.grid[row][col] == 15:
            x = col * self.block_size
            y = row * self.block_size
            self.draw_rect(x + thick, y + thick,
                           self.block_size - (2 * thick),
                           self.block_size - (2 * thick),
                           0xFFD3D3D3)
```

Os dois ciclos for aninhados do grid

for row in range(len(self.grid)) percorre os índices 0, 1, 2, 3, 4 — um por cada linha do grid. Dentro desse, for col in range(len(self.grid[row])) percorre 0, 1, 2, 3, 4, 5, 6 — uma por cada coluna. Juntos visitam todas as células do labirinto.

```
# Para um grid 5 linhas × 7 colunas:
# row=0, col=0 → célula (0,0)
# row=0, col=1 → célula (0,1)
# ....
# row=4, col=6 → célula (4,6)    # última célula
```

x = col * self.block_size

```
# col = 3,    block_size = 64
x = 3 * 64 = 192    # píxel X onde começa este bloco no ecrã
```

Esta linha converte a posição da coluna no grid (índice inteiro) para a posição em píxeis no ecrã. A coluna 0 começa no píxel 0, a coluna 1 no píxel 64, a coluna 2 no píxel 128, etc.

y = row * self.block_size

```
# row = 2,    block_size = 64
y = 2 * 64 = 128    # píxel Y onde começa este bloco no ecrã
```

Mesma lógica para a vertical. A linha 0 começa no píxel Y=0, a linha 1 no Y=64, a linha 2 no Y=128, etc.

O recuo geométrico: x + thick e y + thick

```
thick = 4    # espessura da parede em píxeis

# Sem recuo (errado – cinzento colaria à parede):
```

```
draw_rect(x, y, block_size, block_size, cinzento)

# Com recuo (correto – cinzento fica dentro das paredes):
draw_rect(x + thick, y + thick, ...)
#          ^^^^^^^^^^^  ^^^^^^^^^^^
#          afasta 4px   afasta 4px
#          da esq.     do topo
```

parede N (4px)	parede N (4px)	parede N (4px)	parede N (4px)	parede N (4px)
par W		cinz.		par E
par W		x+4,y+4		par E
par W				par E
parede S (4px)	parede S (4px)	parede S (4px)	parede S (4px)	parede S (4px)

O recuo de 4px (thick) garante que o cinzento fica dentro das paredes e não as cobre

block_size - (2 * thick) — a largura/altura do cinzento

```
block_size      = 64
thick           = 4
2 * thick       = 8   # retiramos 4px de cada lado

largura do cinzento = 64 - 8 = 56  píxeis
altura do cinzento  = 64 - 8 = 56  píxeis
```

Se o bloco tem 64px e afastamos 4px de cada lado (esquerda e direita), o cinzento tem $64 - 4 - 4 = 56$ px de largura. O (2 * thick) representa os dois lados — início e fim.

 Sem o recuo, o cinzento cobriria as paredes e o labirinto ficaria todo cinzento sem linhas visíveis. O thick é literalmente o que separa o fundo do decorativo das paredes estruturais.

Secção 5 Layer 2 — O Caminho de Solução (a geometria central)

Código completo da Layer 2

```
pt = self.path_thickness  # = 24 píxeis
half_block = self.block_size // 2  # = 64 // 2 = 32

for i in range(len(self.path) - 1):
    r1, c1 = self.path[i]      # célula atual
    r2, c2 = self.path[i + 1]  # célula seguinte

    cx1 = c1 * self.block_size + half_block
    cy1 = r1 * self.block_size + half_block
    cx2 = c2 * self.block_size + half_block
    cy2 = r2 * self.block_size + half_block
```

```

lx = min(cx1, cx2) - (pt // 2)
ly = min(cy1, cy2) - (pt // 2)
lw = abs(cx1 - cx2) + pt if cx1 != cx2 else pt
lh = abs(cy1 - cy2) + pt if cy1 != cy2 else pt

self.draw_rect(lx, ly, lw, lh, 0xFF289D8C)

```

self.block_size // 2 — divisão inteira

```

self.block_size      = 64
self.block_size // 2 = 32 # metade do bloco, SEM casas decimais

# Por que // e não / ?
64 / 2    → 32.0 (float – mlx_pixel_put não aceita floats!)
64 // 2   → 32   (int – correto para coordenadas de píxeis)

```

O `//` é o operador de divisão inteira em Python. Descarta a parte decimal. Como as coordenadas de píxeis têm de ser números inteiros (não podes pintar meio píxel), usamos sempre `//` para divisões de geometria.

 *Regra de ouro: sempre que calcules uma coordenada de píxel, usa `//` em vez de `/`. Evita erros de tipo e garante píxeis exatos.*

half_block — o centro de uma célula

```

half_block = 32 # = 64 // 2

# A célula na coluna 2 começa no píxel x = 2 * 64 = 128
# O centro dessa célula está em: 128 + 32 = 160

# Geometricamente:
# |<----- 64px ----->|
# |   32px   |   32px   |
# |<----->|<----->|
#           ^
#       centro = início + half_block

```

`half_block` é usado para calcular o ponto central de cada célula do labirinto. Em vez de ligar os cantos das células, o caminho liga os centros — o que produz linhas contínuas e centradas nos corredores.

r1, c1 = self.path[i] — desempacotar coordenadas

```

self.path = [(0,0), (1,0), (1,1), (1,2), (2,2)]

# Quando i = 1:
self.path[1] → (1, 0) # é um par (row, col)

```

```

r1, c1 = (1, 0)
# r1 = 1   (linha 1)
# c1 = 0   (coluna 0)

self.path[2] → (1, 1)
r2, c2 = (1, 1)
# r2 = 1   (linha 1)
# c2 = 1   (coluna 1)

```

O path armazena pares (row, col). Python permite 'desempacotar' um tuplo em variáveis separadas numa só linha. `r1,c1 = self.path[i]` é equivalente a `r1 = self.path[i][0]` e `c1 = self.path[i][1]`.

cx1, cy1, cx2, cy2 — centros em píxeis

```

half_block = 32

# Célula (r1=1, c1=0) → ponto central em píxeis:
cx1 = c1 * block_size + half_block = 0 * 64 + 32 = 32
cy1 = r1 * block_size + half_block = 1 * 64 + 32 = 96

# Célula (r2=1, c2=1) → ponto central em píxeis:
cx2 = c2 * block_size + half_block = 1 * 64 + 32 = 96
cy2 = r2 * block_size + half_block = 1 * 64 + 32 = 96

# Os dois centros: (32, 96) → (96, 96) — movimento horizontal!

```

`c` (de column) $\times$ `block_size` dá o píxel X do início da célula. Somamos `half_block` para chegar ao centro. O prefixo `c` antes de `x/y` indica 'center' — `cx` = center x, `cy` = center y.

lx, ly, lw, lh — o retângulo que liga os centros

```

# Continuando o exemplo: (cx1=32, cy1=96) → (cx2=96, cy2=96)
# É um segmento horizontal (cy1 == cy2)
pt = 24 # espessura do caminho

lx = min(32, 96) - (24 // 2) = 32 - 12 = 20
ly = min(96, 96) - (24 // 2) = 96 - 12 = 84

# lw: cx1 != cx2 (movimento horizontal)
lw = abs(32 - 96) + 24 = 64 + 24 = 88

# lh: cy1 == cy2 (sem movimento vertical)
lh = pt = 24

# Resultado: draw_rect(20, 84, 88, 24, cor_teal)

```

Porquê min() e não o valor diretamente?

```

# O path pode ir em qualquer direção:

```



```
# Da célula (1,2) para (1,1) = movimento para a ESQUERDA
# cx1 = 160, cx2 = 96

# Se usássemos cx1 diretamente como lx:
lx = cx1 - (pt // 2) = 160 - 12 = 148 # ERRADO! Começa à direita

# Com min():
lx = min(160, 96) - 12 = 96 - 12 = 84 # CORRETO! Começa na esquerda
```

`min(cx1, cx2)` garante que `lx` é sempre o píxel mais à esquerda dos dois centros, independentemente da direção do movimento. Sem isto, o retângulo seria desenhado fora do lugar quando o caminho anda para trás.

`abs(cx1 - cx2)` — a distância entre centros

```
# Movimento horizontal: cx1=32, cx2=96
abs(32 - 96) = abs(-64) = 64 # distância = 64px = 1 bloco

# Adicionamos pt=24 para que a linha 'toque' em ambos os centros:
lw = 64 + 24 = 88

# Porquê + pt?
# O retângulo começa 12px ANTES do centro esquerdo (- pt//2)
# e termina 12px DEPOIS do centro direito (+ pt//2 implícito)
# Isso garante que a linha cobre completamente os dois pontos.
```

`abs()` (valor absoluto) garante que a distância é sempre positiva, independentemente da direção do segmento. Sem `abs()`, `cx1-cx2` poderia ser negativo e dar uma largura negativa ao `draw_rect` — que desenharia nada.

`cx1 != cx2 else pt` — movimento vertical vs horizontal

```
lw = abs(cx1 - cx2) + pt if cx1 != cx2 else pt
# |__ distância H __|      ^ H?      ^ V → largura = só pt

lh = abs(cy1 - cy2) + pt if cy1 != cy2 else pt
# |__ distância V __|      ^ V?      ^ H → altura = só pt

# Caso H (esq-dir): lw = 64+24=88, lh = 24
# Caso V (cima-baixo): lw = 24, lh = 64+24=88
```

Esta é uma expressão condicional inline (ternário Python). Se os centros têm o mesmo X (`cx1==cx2`), o segmento é vertical e a largura é apenas a espessura `pt`. Se têm o mesmo Y (`cy1==cy2`), é horizontal e a altura é apenas `pt`. Garante que o retângulo é sempre um segmento fino e não um bloco quadrado.

 *O path deste labirinto só tem movimentos horizontal ou vertical (não diagonal). Por isso, sempre uma das condições é verdadeira e a outra falsa.*



Secção 6 Layer 3 — Marcadores de Entrada e Saída

Código da Layer 3

```

half_block = self.block_size // 2    # = 32

start_row, start_col = self.start    # ex: (0, 0)
st_cx = start_col * self.block_size + half_block    # cx do start
st_cy = start_row * self.block_size + half_block    # cy do start
self.draw_rect(st_cx - 24, st_cy - 24, 48, 48, 0xFF00FF00)    # verde

end_row, end_col = self.end          # ex: (2, 2)
ed_cx = end_col * self.block_size + half_block
ed_cy = end_row * self.block_size + half_block
self.draw_rect(ed_cx - 24, ed_cy - 24, 48, 48, 0xFFFF0000)    # vermelho

```

st_cx e ed_cx — centros do start e end

st_cx e ed_cx são os píxeis X dos centros das células de início e fim. O prefixo st_ é de 'start', ed_ é de 'end'. A fórmula é idêntica à do caminho: $\text{coluna} \times \text{block_size} + \text{half_block}$.

st_cx - 24 e st_cy - 24 — centrar o marcador

```

# O marcador tem 48×48 píxeis.
# Para o centrar no ponto (st_cx, st_cy):

# início do marcador = centro - metade do tamanho
start_x = st_cx - 24    # = st_cx - (48 // 2)
start_y = st_cy - 24    # = st_cy - (48 // 2)

# Exemplo: st_cx = 32, st_cy = 32
draw_rect(32-24, 32-24, 48, 48, verde)
# = draw_rect(8, 8, 48, 48, verde)
# O bloco verde vai de (8,8) a (55,55) — centrado em (32,32) ✓

```

Para centrar um retângulo de tamanho W numa posição cx, o canto esquerdo tem de ser $cx - W/2$. Como o marcador tem 48px, subtraímos 24 ($= 48 \div 2$). Sem esta subtração, o marcador começaria no centro da célula e ficaria deslocado para baixo-direita.



Repara que $24 = 48 // 2 = \text{tamanho do marcador a dividir por 2}$. Se quiseres um marcador maior ou menor, bastava mudar os 48 e os 24 proporcionalmente.



Secção 7 Layer 4 — As Paredes e a Álgebra de Bits

Código da Layer 4

```
current_color = self.wall_colors[self.color_index]

for row in range(len(self.grid)):
    for col in range(len(self.grid[row])):
        x = col * self.block_size
        y = row * self.block_size
        cell_value = self.grid[row][col]

        if cell_value & 1:  # parede Norte
            self.draw_rect(x, y, self.block_size, thick, current_color)
        if cell_value & 2:  # parede Este
            self.draw_rect(x + self.block_size - thick, y, thick,
self.block_size, current_color)
        if cell_value & 4:  # parede Sul
            self.draw_rect(x, y + self.block_size - thick,
self.block_size, thick, current_color)
        if cell_value & 8:  # parede Oeste
            self.draw_rect(x, y, thick, self.block_size, current_color)
```

cell_value = self.grid[row][col]

```
# Para a célula na linha 0, coluna 0:
cell_value = self.grid[0][0] = 15

# 15 em binário de 4 bits:
15 = 0b1111
#      ||||
#      |||+-- bit 0 = 1 → parede Norte
#      ||+--- bit 1 = 1 → parede Este
#      |+---- bit 2 = 1 → parede Sul
#      +----- bit 3 = 1 → parede Oeste
#
# Valor 15 = todas as paredes ativas (célula completamente fechada)
```

O operador & (bitwise AND) — extrair um bit

```
# cell_value = 6 = 0b0110

cell_value & 1:  0110 & 0001 = 0000 = 0 → FALSO (sem parede Norte)
cell_value & 2:  0110 & 0010 = 0010 = 2 → VERDADE (parede Este)
cell_value & 4:  0110 & 0100 = 0100 = 4 → VERDADE (parede Sul)
cell_value & 8:  0110 & 1000 = 0000 = 0 → FALSO (sem parede Oeste)

# Em Python: qualquer valor != 0 é 'True' no if
```

Geometria das 4 paredes — onde fica cada draw_rect

Variável	Exemplo (7×5)	O que significa
Parede N	<code>draw_rect(x,</code>	Faixa horizontal no TOPO do bloco. Começa em (x,y),

	<code>y, block, thick, cor)</code>	tem largura=bloco inteiro, altura=thick
Parede E	<code>draw_rect(x+block-thick, y, thick, block, cor)</code>	Faixa vertical na DIREITA. $x+block-thick$ = último 'thick' píxeis do lado direito
Parede S	<code>draw_rect(x, y+block-thick, block, thick, cor)</code>	Faixa horizontal em BAIXO. $y+block-thick$ = últimas linhas do bloco
Parede O	<code>draw_rect(x, y, thick, block, cor)</code>	Faixa vertical na ESQUERDA. Começa em (x,y), tem largura=thick, altura=bloco inteiro

x + block_size - thick — a parede Este

```
# block_size = 64, thick = 4
# A parede Este é os últimos 4 píxeis do lado direito do bloco.

# O bloco começa em x e termina em x + 64.
# Os últimos 4 píxeis estão de x+60 até x+63:

x + block_size - thick = x + 64 - 4 = x + 60

# draw_rect(x+60, y, 4, 64, cor) → faixa de 4px no lado direito ✓
```

Usamos subtração porque queremos os últimos thick píxeis do bloco, não os primeiros. Se usássemos $x + block_size$, começaria fora do bloco (no bloco seguinte).

y + block_size - thick — a parede Sul

```
# Mesma lógica, mas vertical.
# A parede Sul são os últimos 4 píxeis no fundo do bloco.

y + block_size - thick = y + 64 - 4 = y + 60

# draw_rect(x, y+60, 64, 4, cor) → faixa de 4px no fundo ✓
```



Secção 8 Síntese — Um Bloco de 64px Completo

Aqui está como todos os cálculos se combinam num único bloco de 64×64 píxeis (exemplo: coluna 2, linha 1):

```
col=2, row=1, block_size=64, thick=4, half_block=32

x = 2 * 64 = 128    (píxel X do canto superior esquerdo)
y = 1 * 64 = 64     (píxel Y do canto superior esquerdo)
```

```

— Layer 1 (fundo cinzento, se valor==15) —————
draw_rect(128+4, 64+4, 64-(2*4), 64-(2*4), cinzento)
          = (132, 68, 56, 56, cinzento)

— Layer 3 (se for célula start) —————
cx = 128 + 32 = 160
cy = 64 + 32 = 96
draw_rect(160-24, 96-24, 48, 48, verde)
          = (136, 72, 48, 48, verde)

— Layer 4 (parede Este, se bit 2 ativo) —————
draw_rect(128+64-4, 64, 4, 64, cor_parede)
          = (188, 64, 4, 64, cor_parede)

```

💡 Quando o avaliador perguntar qualquer coisa sobre geometria, substitui números concretos na fórmula e mostra o resultado. Isso prova que entendes o mecanismo, não só memorizaste o código.

⚡ Secção 9 Referência Rápida — Todas as Variáveis

Variável / Expressão	Valor (7×5, b=64)	Significado
len(self.grid)	5	Número de linhas (altura do labirinto em células)
len(self.grid[0])	7	Número de colunas (largura do labirinto em células)
self.block_size	64	Largura e altura de cada célula em píxeis
window_width	448	Largura total da janela: 7 × 64 píxeis
window_height	320	Altura total da janela: 5 × 64 píxeis
thick	4	Espessura das paredes em píxeis
half_block	32	Metade de um bloco: 64 // 2 — o centro de uma célula
x = col * block_size	128 (col=2)	Píxel X do canto esq. da célula na coluna col
y = row * block_size	64 (row=1)	Píxel Y do canto sup. da célula na linha row
cx = col*bs + half_block	160 (col=2)	Píxel X do centro da célula
cy = row*bs + half_block	96 (row=1)	Píxel Y do centro da célula
lx = min(cx1,cx2)-(pt//2)	varia	Píxel X do canto esq. do segmento de caminho
ly = min(cy1,cy2)-(pt//2)	varia	Píxel Y do canto sup. do segmento de caminho
lw = abs(cx1-cx2)+pt	varia	Largura do segmento de caminho (+ espessura pt)
lh = abs(cy1-cy2)+pt	varia	Altura do segmento de caminho (+ espessura pt)
cell_value & 1	0 ou 1	Bit 0: parede Norte ativa?
cell_value & 2	0 ou 2	Bit 1: parede Este ativa?

<code>cell_value & 4</code>	<code>0 ou 4</code>	Bit 2: parede Sul ativa?
<code>cell_value & 8</code>	<code>0 ou 8</code>	Bit 3: parede Oeste ativa?
<code>x+block_size-thick</code>	<code>x+60</code>	Início da parede Este (últimos 4px do lado direito)
<code>y+block_size-thick</code>	<code>y+60</code>	Início da parede Sul (últimos 4px do fundo)
<code>block_size-(2*thick)</code>	<code>56</code>	Dimensão do fundo cinzento (sem as paredes dos 2 lados)